

Teaching AI the Rules of Finance

Amirkhan Aidarkhan

April 8, 2026

1 Problem Description

Traditionally, training an AI to predict financial outcomes, such as stock prices or option values, relies heavily on large datasets. The model attempts to find patterns within the data, but it often lacks an understanding of the underlying logic. This can result in predictions that are mathematically inconsistent or physically impossible.

In this project, I will implement a **Physics-Informed Neural Network (PINN)**. Unlike standard models, a PINN is constrained by a "cheat sheet" of mathematical rules. In this case specifically, it is constrained by the Black-Scholes-Merton equation. By embedding these rules directly into the neural network's loss function, the AI is forced to satisfy the fundamental logic of arbitrage-free pricing. This approach ensures that the model does not just guess based on data but actually "understands" the financial principles it is simulating. To handle the high-dimensional calculations and simulations, I will utilize an NVIDIA RTX 6000 GPU.

2 Mathematical Framework

The core constraint of the model is the Black-Scholes-Merton partial differential equation. This equation represents a balance between the time-decay of an option, the movement of the underlying asset, and the risk-free interest rate.

The equation is defined as:

$$\frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} + rS \frac{\partial V}{\partial S} - rV = 0 \quad (1)$$

where:

- V : The price of the option.
- S : The current price of the underlying stock.
- t : Time.
- σ : Volatility of the stock.
- r : The risk-free interest rate.

During training, the AI calculates its own "physics loss" by checking if its predicted V satisfies this equation. If the left side of the equation does not equal zero, the AI penalizes itself and adjusts its internal weights until the mathematics are satisfied.

3 Project Goals

1. Program a custom loss function in a deep learning framework (such as PyTorch or JAX) that penalizes the model whenever it violates the Black-Scholes PDE.
2. Train the model on European Call Options and compare the model's predicted prices against the mathematically proven Black-Scholes formula to verify accuracy (basically benchmarking a model).
3. Generate 3D visualizations showing the option price surface across different stock prices and time-to-maturities to demonstrate the model's stability.